

DIAGRAMAS DEL PROYECTO

Sistema de Detección de Intrusiones
para PyMEs del Estado Barinas

Equipo de Desarrollo

Isaac Silva | Carlos Herrera | Ángel Ramos

Fecha: 23 de April de 2026

ÍNDICE DE DIAGRAMAS

Diagrama	Descripción	Página
1. Diagrama de Gantt	Cronograma de 9 semanas	3
2. Arquitectura del Sistema	5 capas del sistema	5
3. Flujo de Detección	Proceso de detección de amenazas	7
4. Metodología Agile/Scrum	Ciclo de desarrollo semanal	9
5. Ecosistema de IA	Herramientas de soporte	11

1. DIAGRAMA DE GANTT - CRONOGRAMA

El diagrama de Gantt visualiza la planificación completa del proyecto a lo largo de 9 semanas, distribuidas en sprints de desarrollo ágil. Cada desarrollador tiene asignado un color específico que facilita el seguimiento de sus responsabilidades.

Código de Colores por Desarrollador:

Desarrollador	Módulo Principal	Color
Isaac Silva	Captura y Detección de Red	■ Amarillo
Carlos Herrera	Backend API y Notificaciones	■ Verde
Ángel Ramos	Base de Datos y Frontend	■ Azul
Equipo Completo	Testing y Documentación	■ Gris

Desglose por Sprints:

Sprint	Semanas	Objetivos Principales	Entregable
1-2	S1-S2	Setup técnico: Scapy, FastAPI, Supabase	Infraestructura funcionando
3-4	S3-S4	Motor de detección: 4 módulos + API REST	Detecta 4 amenazas + Dashboard
5-6	S5-S6	Alertas: Telegram + Email + Gráficas	Notificaciones en tiempo real
7-8	S7-S8	Testing de carga + Optimización	Sistema estable y robusto
9	S9	Documentación + Video demo	MVP completo presentable

Características Clave del Cronograma:

- **División modular:** Cada desarrollador trabaja en un módulo independiente para minimizar conflictos
- **Entregables semanales:** Cada sprint termina con funcionalidad demostrable
- **Testing progresivo:** Validación continua evita crisis de última hora
- **Flexibilidad:** Los sprints cortos permiten ajustes rápidos según necesidad

2. ARQUITECTURA DEL SISTEMA - 5 CAPAS

La arquitectura del sistema está organizada en 5 capas funcionales que procesan el tráfico de red desde su captura hasta la generación de alertas para el administrador. Cada capa tiene responsabilidades específicas y se comunica con las capas adyacentes mediante interfaces bien definidas.

Descripción de Capas:

Capa	Tecnología	Función Principal
1. Captura	Scapy	Escucha interfaces de red en modo promiscuo, captura paquetes TCP/UDP/ICMP
2. Análisis	Python + Reglas	Aplica módulos de detección: escaneos, fuerza bruta, IPs maliciosas, DoS
3. Persistencia	Supabase PostgreSQL	Almacena eventos, alertas y configuraciones en base de datos
4A. Notificaciones	Telegram + SMTP	Envía alertas automáticas vía Bot de Telegram y correo electrónico
4B. Interfaz	FastAPI + HTML/JS	API REST para consultas + Dashboard web de administración

Flujo de Datos:

- Paso 1:** Red local → Scapy captura paquetes
Paso 2: Motor de análisis evalúa contra reglas de detección
Paso 3: Si hay amenaza → Registro en Supabase
Paso 4: Generación de alerta con severidad y descripción
Paso 5: Bifurcación: Telegram Bot + Email SMTP (paralelo)
Paso 6: Dashboard consulta eventos vía API para visualización

Beneficios de esta Arquitectura:

- **Modularidad:** Cada capa puede desarrollarse y probarse independientemente
- **Escalabilidad:** Fácil agregar nuevos módulos de detección o canales de notificación
- **Mantenibilidad:** Separación clara de responsabilidades facilita debugging
- **Desempeño:** Procesamiento asíncrono en FastAPI evita bloqueos

3. FLUJO DE DETECCIÓN DE AMENAZAS

Este diagrama ilustra el proceso completo desde que un paquete de red llega al sistema hasta que se envía una alerta al administrador. El sistema está diseñado para responder en menos de 5 segundos desde la detección hasta la notificación.

Proceso Detallado:

Paso	Acción	Tiempo Estimado
1	Paquete llega → Scapy captura y decodifica protocolos	< 100ms
2	Motor evalúa 4 módulos de detección en paralelo	< 500ms
3	¿Amenaza detectada? → Decisión binaria	Instantáneo
4	SÍ → Registro en Supabase (IP, tipo, timestamp)	< 200ms
5	Generación de alerta con severidad y descripción	< 100ms
6	Envío paralelo: Telegram Bot + Email SMTP	< 2s
7	Confirmación: Alerta entregada exitosamente	< 1s

Módulos de Detección Implementados:

- 1. Escaneo de Puertos:** Detecta intentos de mapeo de red (SYN scan, port sweep)
- 2. Fuerza Bruta:** Identifica intentos repetidos de acceso fallido (SSH, FTP, HTTP)
- 3. IPs Maliciosas:** Compara contra blacklists conocidas de IPs sospechosas
- 4. Denegación de Servicio:** Detecta patrones de flood (ICMP flood, SYN flood)

Notas Importantes:

- Si NO hay amenaza, el paquete se descarta y el sistema continúa monitoreando
- Las alertas incluyen: IP origen, tipo de amenaza, timestamp, severidad (baja/media/alta)
- El sistema mantiene histórico completo en Supabase para análisis posterior
- Telegram y Email se envían en paralelo para redundancia

4. METODOLOGÍA AGILE/SCRUM ADAPTADA

El proyecto utiliza una metodología Agile/Scrum híbrida adaptada para equipos pequeños (3 personas) con sprints semanales. Esta estructura permite iteraciones rápidas, testing continuo y ajustes ágiles ante bloqueos técnicos o cambios de alcance.

Ciclo Semanal de Desarrollo:

Día	Actividad	Duración	Participantes
Lunes	Daily Stand-up + Planning	30 minutos	Todo el equipo
Mar-Jue	Desarrollo modular paralelo	3 días	Individual por módulo
Viernes	Sprint Review + Retrospectiva	2 horas	Todo el equipo
Sábado	Testing de integración	Medio día	Todo el equipo
Domingo	Cierre de sprint + Commit	2-3 horas	Todo el equipo

Ceremonias Scrum Adaptadas:

Daily Stand-up (Lunes, 30min):

- ¿Qué logré la semana pasada?
- ¿Qué haré esta semana?
- ¿Qué bloqueos tengo?

Sprint Review (Viernes):

- Demo de funcionalidad completada
- Validación contra criterios de aceptación
- Feedback del equipo

Retrospectiva (Viernes):

- ¿Qué funcionó bien?
- ¿Qué mejorar?
- Acciones concretas para próximo sprint

Beneficios de esta Metodología:

- **Flexibilidad:** Sprints cortos permiten pivotar rápido ante obstáculos
- **División modular:** Minimiza conflictos, maximiza paralelización
- **MVP primero:** Enfoque en funcionalidad core, optimización después
- **Soporte continuo IA:** Claude Code, Gemini, GPT y Ollama disponibles 24/7

5. ECOSISTEMA DE HERRAMIENTAS IA

El equipo cuenta con 4 herramientas de inteligencia artificial que aceleran el desarrollo, mejoran la calidad del código y resuelven bloqueos técnicos. Cada IA tiene especialización específica que complementa las habilidades del equipo humano.

Herramientas y Sus Casos de Uso:

Herramienta	Especialización	Casos de Uso Principales
Claude Code	Refactoring y Arquitectura	• Crear módulos Scapy desde cero • Diseñar arquitectura de componentes • Revisar código Python
Gemini 3.1	Análisis de Patrones	• Mejorar precisión de detección • Reducir falsos positivos • Análisis de comportamiento de red
GPT-4	Debugging Crítico	• Resolver bugs complejos • Optimización de rendimiento • Explicaciones técnicas detalladas
Ollama (Local)	Análisis Offline	• Testing sin conexión a internet • Desarrollo en campo • Privacidad de código sensible

Workflow de Uso Integrado:

Fase 1 - Arquitectura (Claude Code):

Diseñar estructura de módulos, decidir patrones de diseño, crear esqueletos de clases

Fase 2 - Desarrollo (Claude Code + Gemini):

Implementar lógica de negocio, refinar algoritmos de detección con Gemini

Fase 3 - Debugging (GPT-4):

Resolver errores complejos, optimizar cuellos de botella de rendimiento

Fase 4 - Testing (Ollama):

Validación offline, pruebas en redes aisladas sin acceso a internet

Consejos para Uso Efectivo de IA:

- **Contexto claro:** Proporcionar código completo, mensajes de error exactos
- **Iteración:** Refinar prompts basándose en respuestas previas
- **Validación humana:** Siempre revisar código generado por IA antes de commit
- **Documentación:** Pedir a la IA que genere docstrings y comentarios